ICMC USP SÃO CARLOS

USP Universidade de São Paulo



IA, Otimização e IoT para Indústrias e Negócios
Do Zero ao MLOps

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT

Módulo 1

Fundamentos de Dados e Aprendizado de Máquina

Aplicado a Operações e Negócios

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Validação e Generalização

Sumário

01 O que é Estratégia de Validação e Generalização?

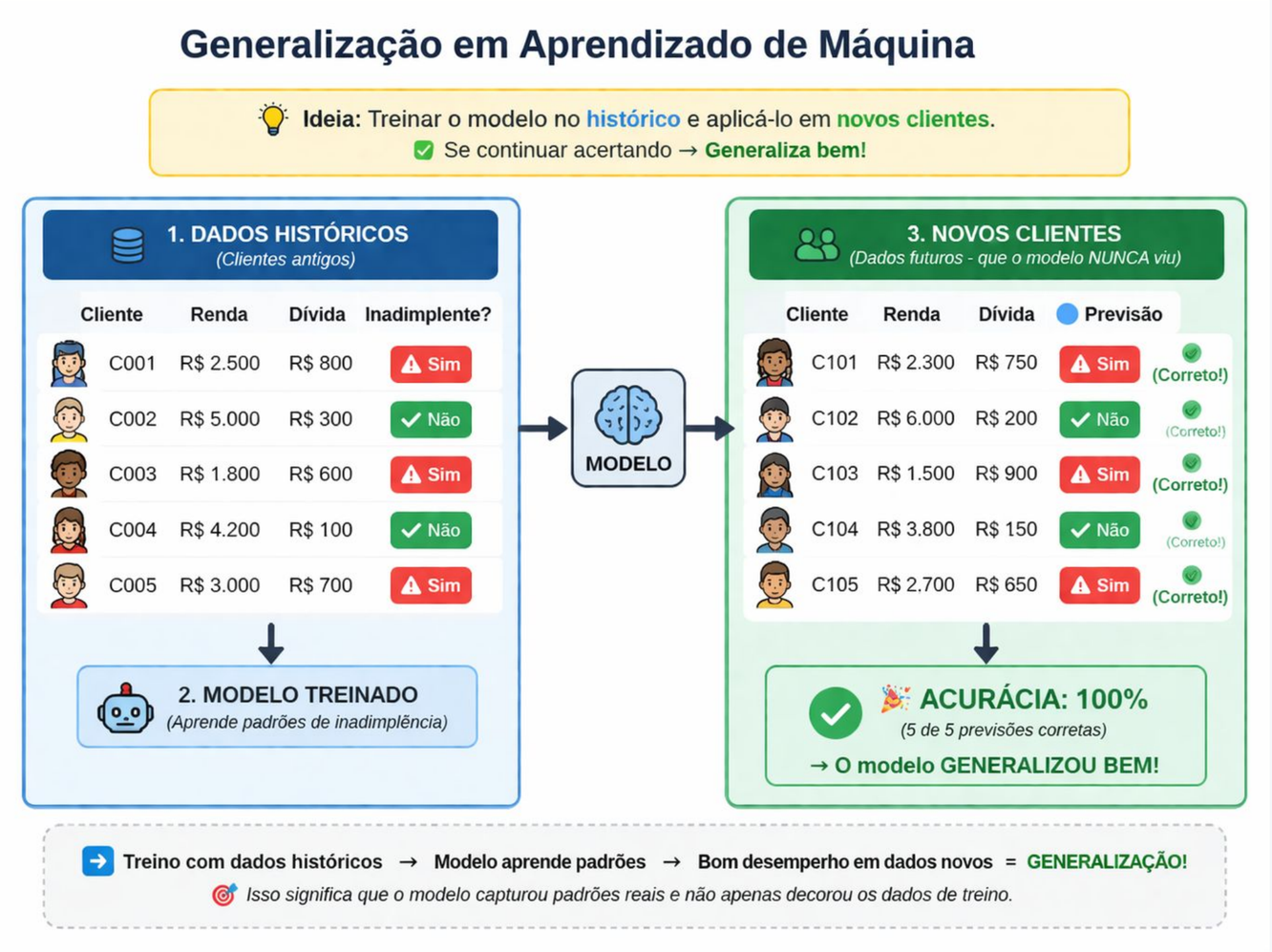
02 Hold-out

03 K-fold cross-validation

04 Underfitting e Overfitting

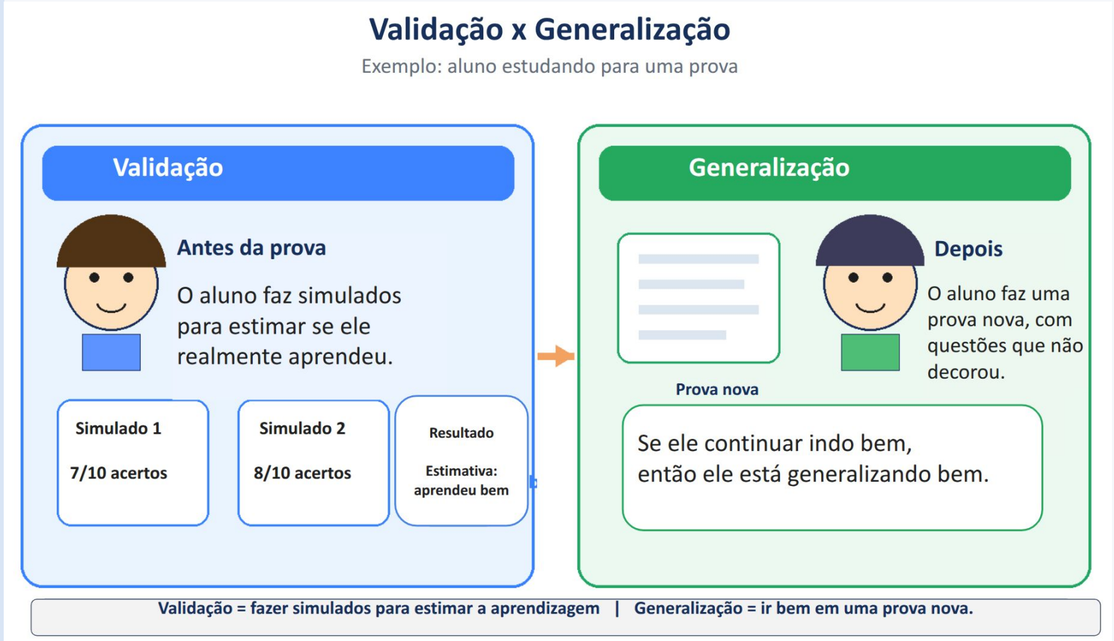
Estratégias de validação e generalização são procedimentos usados para responder à pergunta mais importante de um modelo preditivo: **“ele vai funcionar bem em dados não vistos durante o treinamento?”**

Um modelo generaliza bem quando **aprende padrões** que se repetem na **população**, ou seja, tem bom desempenho fora da amostra, e não só nos dados usados para ajustá-lo.



O que é Estratégia de Validação e Generalização?

Validação é o procedimento usado para estimar se essa generalização será boa.



Forma simples de entender

➔ generalização: propriedade desejada do modelo;

➔ validação: método usado para avaliar essa propriedade.

- Se o modelo vai muito bem no treino, mas mal em dados novos, ele não generaliza bem.
- Para descobrir isso antes, usamos estratégias de validação:
  - hold-out
  - k-fold cross-validation
- **Validação** se torna o processo de teste controlado.
- **Generalização** se torna o resultado que queremos: bom desempenho fora da amostra.

Consiste em separar os dados em partes distintas, normalmente um conjunto para **treino** e outro para **teste.**

Treino
- Geralmente 70-80% dos dados.
- O modelo aprende aqui.

Teste
- geralmente 20-30% dos dados.
- É o "simulado" que só vemos ao final.

- Exemplo: Você tem 1.000 e-mails, metade spam, metade não spam.
  1. Embaralha os e-mails.
  2. Separa 800 para treino e 200 para teste.
  3. Treina o classificador com os 800.
  4. Ao final, mostra os 200 de teste que ele nunca viu.
  5. Se acertar 95% dos 200, essa é sua estimativa de acurácia.

Vantagens
- Simples, rápido, computacionalmente barato.
- Bom quando temos muitos dados.

Desvantagens
- A estimativa de desempenho pode variar muito dependendo de quais exemplos foram para o teste.
- Se os dados forem poucos, jogar 30% fora para teste pode significar perder exemplos raros importantes.

Em uma versão um pouco mais cuidadosa, usa-se treino, **validação** e teste

Treino
- Treina-se no conjunto de treino.
- Geralmente 70% dos dados.

Validação
- Escolhem-se hiperparâmetros no conjunto de validação.
- Geralmente 15% dos dados.

Teste
- Mede-se o desempenho final no teste.
- Geralmente 15% dos dados.

Lógica empregada:

1. Ajustar o modelo nos 70%;
2. Comparar alternativas nos 15% de validação;
3. Depois de decidir o modelo final, medir uma única vez nos 15% de teste.

Vantagens
- Evita que o teste participe da tomada de decisão

Desvantagens
- Ao dividir os dados em três partes você reduz o número de exemplos disponíveis para aprender, e o resultado pode depender bastante da divisão aleatória feita.

O conjunto de treino é dividido em k partes, onde o modelo é treinado em k−1 partes e validado na restante.

Funcionamento

1. Dividimos os dados em k partes (folds) de tamanho igual
2. Repetimos o processo k vezes:
   Usamos k-1 partes para treino.
   A parte restante para teste (validação).
1. Calculamos a métrica (acurácia, erro) em cada uma das k iterações.
2. Ao final, tiramos a média e o desvio padrão dos k resultados.

Exemplo: Temos 1.000 e-mails.

Fold 1: 200 e-mails, Fold 2: 200 e-mails, Fold 3: 200 e-mails

Fold 4: 200 e-mails, Fold 5: 200 e-mails

- Iteração 1: Treina com folds 2,3,4,5 (800 e-mails) e testa no fold 1 (200) → acurácia 94%
- Iteração 2: Treina com 1,3,4,5 e testa no 2 → 93%
- ...
- Iteração 5: Treina com 1,2,3,4 e testa no 5 → 95%

Resultado final: 94% ± 0,8% de acurácia.

Vantagens

- Usa todos os dados para treino em algum momento.
- Mais robusta: a média reduz a variabilidade.
- O desvio padrão mostra se o modelo é estável.

Desvantagens

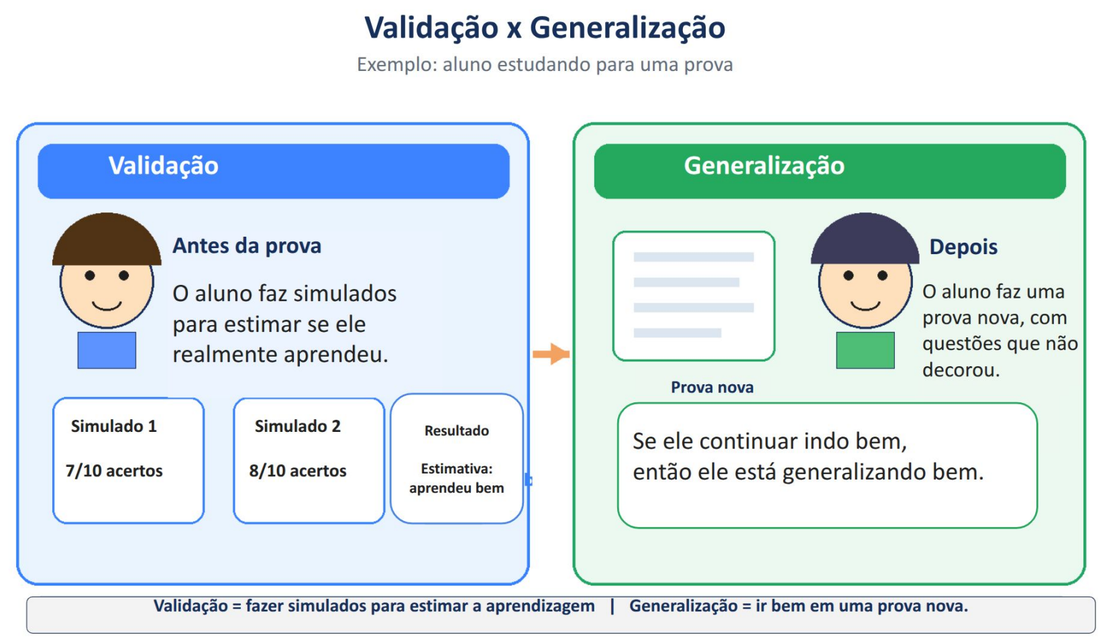
- Custosa computacionalmente (treina k modelos).
- Pode ser excessiva para datasets muito grandes (hold-out já é suficiente).

A validação (hold-out ou k-fold) é a ferramenta que nos permite estimar a capacidade de generalização sem contaminar o modelo com os dados de teste.

Validação cruzada (k-fold cross validation) costuma ser melhor que hold-out:

- Usa dados de forma mais eficiente e reduz a dependência de uma única amostra de validação.
- Em vez de perguntar “como foi o desempenho nesta divisão específica?”, ela pergunta “como o desempenho se comporta em várias divisões plausíveis dos dados?”.
- Um forma de tratar a instabilidade de uma única validação é usar ***repeated cross-validation***:
  - múltiplas validações em subconjuntos diferentes, reforçando a ideia de que generalização não deve ser julgada por um único número isolado.

Underfitting e Overfitting



- **Underfitting (subajuste)**: O aluno não aprendeu nem o conteúdo do simulado. Erra muito no treino e na prova.
- **Overfitting (sobreajuste)**: O aluno decorou todas as respostas do simulado, mas não entendeu o conteúdo. Vai muito bem no treino, mas mal na prova.
- Boa generalização: O aluno entendeu os conceitos. Vai bem no treino e na prova (dados novos).

Overfitting: quando o modelo se ajusta demais aos dados de treino e vai mal em dados novos.

Underfitting: quando o modelo é simples demais e não aprende bem nem os dados de treino.

Model complexity (Complexidade do Modelo) é o “grau de flexibilidade” do modelo.

- Um modelo de baixa complexidade é mais rígido, simples, com pouca capacidade de se adaptar aos dados.
- Um modelo de alta complexidade é mais flexível, consegue se ajustar a padrões mais detalhados.

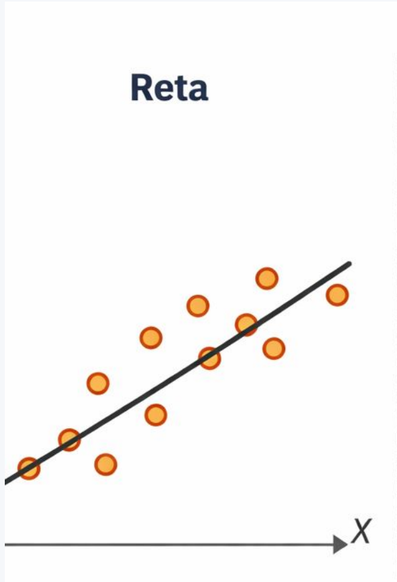
Exemplos:

Uma reta ajustando dados curvos

→ baixa complexidade.

Um polinômio de grau muito alto

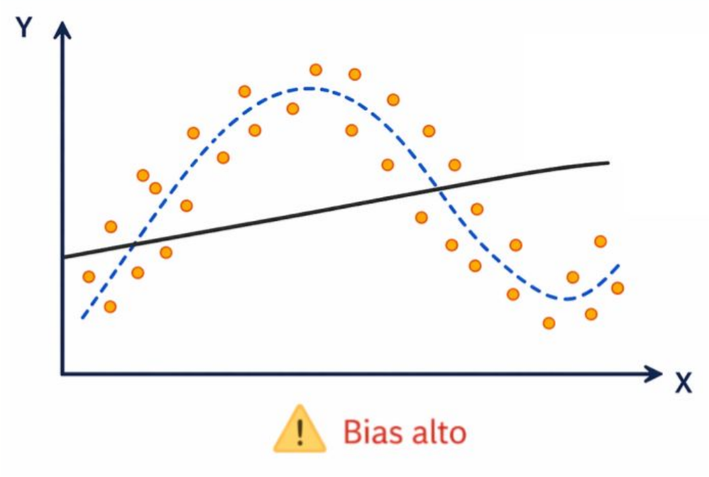
→ alta complexidade.





A complexidade influencia diretamente bias e variance.

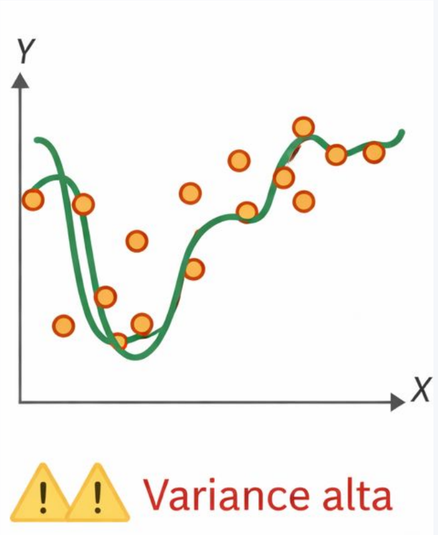
Bias é a tendência do modelo errar sistematicamente ao fazer suposições simplificadoras demais sobre a relação entre entrada e saída.



O modelo faz suposições muito fortes, errando sistematicamente.

- Continua cometendo erros parecidos mesmo com aumento no número de dados.
- Bias alto → modelo tende a underfitting.
- O modelo não aprende nem o padrão principal dos dados.

Variance (variância) é o erro causado pelo modelo ser sensível demais às pequenas variações do conjunto de treino.



Modelo muda muito quando mudamos um pouco os dados de treino.

- Aprende detalhes específicos, ruído e acidentes da amostra.
- Parece ótimo no treino, mas piora em dados novos.

- Variância alta → modelo tende a overfitting.
- O modelo aprende demais o treino e generaliza mal.

Temos uma relação entre bias, variance, complexidade do modelo, underfitting e overfitting

À medida que a complexidade do modelo aumenta:

- Bias tende a diminuir, porque o modelo fica mais capaz de representar padrões complexos.
- Variância tende a aumentar, porque o modelo fica mais sensível ao ruído dos dados.

Trade-off:

- modelo muito simples → bias alto, variância baixa;
- modelo muito complexo → bias baixo, variância alta;
- bom modelo → equilíbrio entre os dois.

Baixa complexidade:

- “não consigo representar o padrão” (underfitting)

Alta complexidade:

- “represento até o ruído” (overfitting)

Boa complexidade

- “represento o padrão sem decorar o ruído”

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Referências

- Faceli, Katti, et al. "Inteligência artificial: uma abordagem de aprendizado de máquina." (2021).
- Hastie, T.; Tibshirani, R.; Friedman, J. Model Assessment and Selection, capítulo de The Elements of Statistical Learning (2009).
- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3ª ed. (2022).





Obrigado!

opaiot@icmc.usp.br
+55 (16) 99962-7637

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT